

Maximal Semantic Cliques Mining in Vector-Attributed Graphs

Tianyi Gao

Beijing Institute of Technology
Beijing, China
tygao@bit.edu.cn

Hongchao Qin

Beijing Institute of Technology
Beijing, China
hcqin@bit.edu.cn

ABSTRACT

Maximal clique enumeration (MCE) identifies the densest structural subgraphs, yet in vector-attributed graphs the resulting cliques may group semantically heterogeneous vertices. We introduce the *average-constrained maximal semantic clique* (ACMSC) problem, which requires every output clique to achieve an average pairwise cosine similarity above a threshold τ . The average constraint is a global, non-monotone property that invalidates the structural invariants underpinning classical MCE algorithms, most notably pivot-based pruning. We first derive SEMBK, a correct but pivot-free baseline that exposes the fundamental difficulty. We then propose two complementary algorithms. STRSUB decouples structural and semantic search: it applies the full Tomita pivot in a τ -agnostic structural phase, then prunes subset-size layers via a novel Top-Edge upper bound. MONOSEMMCE departs from Bron-Kerbosch entirely and exploits the *Monotone Path Property*—every ACMSC admits a non-increasing average-similarity insertion ordering—to define a feasibility window that enables $O(1)$ suffix-level pruning. Experiments on ten real-world graphs (up to 2.4M vertices and 61M edges) show that MONOSEMMCE achieves up to 100 $\times$ speedup at high τ , while STRSUB dominates at low τ ; together they cover the full operating range.

PVLDB Reference Format:

Tianyi Gao and Hongchao Qin. Maximal Semantic Cliques Mining in Vector-Attributed Graphs. PVLDB, 14(1): XXX-XXX, 2025.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Graphs with rich vertex attributes are ubiquitous in modern data management and analytics. Social networks associate users with profile embeddings, knowledge graphs attach entity descriptions to nodes, and biological networks annotate proteins with functional vectors. In all these settings, the attribute space carries semantic meaning that is largely orthogonal to the topological structure: two vertices may be structurally adjacent yet semantically distant, and vice versa. Identifying cohesive subgraphs that are simultaneously structurally dense and semantically coherent is therefore a recurring need across domains.

The maximal clique is a classic structural model of cohesion: every pair of vertices is connected, guaranteeing the highest possible edge density. However, in vector-attributed graphs, a clique may group vertices that are heterogeneous in the attribute space. Conversely, attribute-based community detection methods (e.g., modularity optimization with attribute homogeneity) often sacrifice structural guarantees and produce loosely connected groups. This tension motivates a joint model that enforces both structural completeness and semantic coherence.

The ACMSC Problem. We study the problem of enumerating *average-constrained maximal semantic cliques* (ACMSCs) in a vector-attributed graph. Each vertex carries a real-valued attribute vector; pairwise semantic similarity is measured by the cosine similarity of these vectors. Given a threshold τ , a clique C is *average-constrained* if the average pairwise similarity among its vertices is at least τ . An ACMSC is an average-constrained clique that cannot be extended by any additional vertex while preserving both the clique property and the average constraint. The task is to enumerate *all* ACMSCs of the input graph.

This formulation has two salient features. First, the average constraint is a *global* property of the clique: it depends on all $\binom{|C|}{2}$ pairwise similarities, not on any single edge. This makes it fundamentally different from edge-level semantic constraints (e.g., requiring every edge to exceed a threshold), which are local and monotone. Second, the average is non-monotone over the subset lattice: adding a vertex can raise or lower the average, so a partial clique that currently violates the constraint may still grow into a valid ACMSC, while one that satisfies it may admit no feasible extension. This non-monotonicity invalidates the structural invariants that underpin the efficiency of classical maximal clique enumeration (MCE) algorithms.

Why Classical MCE Does Not Suffice. The Bron-Kerbosch (BK) algorithm and its modern variants achieve state-of-the-art performance for MCE through two key techniques: *pivot-based pruning* (Tomita et al.) and *degeneracy ordering* (Eppstein et al.). Pivot pruning relies on the fact that structural maximality is closed under supersets: if a vertex u is adjacent to the current pivot p , any maximal clique containing u also contains p , so u can be safely suppressed as an expansion root. Under the average constraint, this closure breaks: a clique containing u but not p may satisfy $\text{avgSim} \geq \tau$ while $\{p, u\}$ does not. Consequently, pivoting is not merely ineffective—it is *incorrect* for ACMSC enumeration. A direct BK adaptation must therefore forgo pivoting entirely and defer all semantic evaluation to the leaves, degenerating to exhaustive structural enumeration regardless of τ .

Our Contributions. We propose two complementary algorithms that address the non-monotonicity challenge from different angles,

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

together covering the full practically meaningful range of thresholds.

- **STRSUB (Structural Decomposition and Subset Enumeration)** resolves the pivot incompatibility by *decoupling* structural and semantic search. Phase 1 enumerates all structural maximal cliques using standard BK with the full Tomita pivot—this phase is τ -agnostic and inherits the $O(dn \cdot 3^{d/3})$ worst-case bound. Phase 2 searches within each structural clique for semantically maximal subsets. A *Top-Edge Layer Pruning* rule computes an upper bound on the best-case average similarity for any size- k subset from the sorted pairwise similarities of the host clique; if the bound falls below τ , the entire size layer is skipped. This makes STRSUB efficient across the full threshold range, with its strongest advantage at low τ where structural and semantic output largely overlap.
- **MONOSEMMCE (Monotone-Path Clique Expansion)** abandons the BK framework entirely and instead integrates τ into the search from the first expansion step. Its foundation is the *Monotone Path Property* (Theorem 4.2): every ACMSC admits a vertex insertion ordering along which the average similarity is non-increasing at every step. This property induces a *sorted-suffix cut* that discards an entire ordered suffix of candidates in $O(1)$ —qualitatively stronger than any per-candidate pruning rule. Combined with a complementary lower-end cut (W -pruning), the two rules define a *feasibility window* over sorted candidates that contracts rapidly as τ increases. MONOSEMMCE expands cliques level-synchronously: layer ℓ holds all live $(\ell+2)$ -cliques satisfying the constraint, each expanded in parallel to produce layer $\ell+1$. Every generated state is semantically live by construction, so no structurally valid but semantically infeasible state is ever created. This makes MONOSEMMCE the algorithm of choice for high- τ regimes, where the feasibility window is narrow and pruning is most aggressive.

Both algorithms are implemented with a common set of engineering optimizations: sparse blocked bitmaps for $O(\text{blocks})$ neighbor intersection, AVX2-accelerated similarity computation, incremental accumulator maintenance to avoid $O(|C|)$ recomputation, and degree-descending BFS vertex reordering for cache locality. Experiments on ten real-world graphs (up to 2.4M vertices and 61M edges) demonstrate that MONOSEMMCE achieves up to 100 $\times$ speedup over the baseline at high τ , while STRSUB dominates at low τ ; together they cover the full operating range. All three semantic algorithms produce identical output, confirming correctness.

Paper Organization. Section 2 formalizes the problem and establishes hardness. Section 3 presents the baseline SEMBK algorithm and explains why pivoting fails. Section 4 develops STRSUB and MONOSEMMCE. Section 5 describes shared implementation optimizations. Section 6 reports experimental results. Section 7 discusses related work, and Section 8 concludes.

2 PRELIMINARIES

2.1 Notations and Problem Definition

We study a *vector-attributed graph* $\mathcal{G} = (V, E, X)$, where V is the vertex set, $E \subseteq \binom{V}{2}$ is the edge set, and $X = \{\mathbf{x}_v \in \mathbb{R}^d \mid v \in V\}$ assigns a d -dimensional attribute vector to each vertex.

DEFINITION 1 (SIMILARITY FUNCTION). A *similarity function* $\text{sim} : \mathbb{R}^d \times \mathbb{R}^d \rightarrow [-1, 1]$ is a symmetric, bounded function measuring the semantic closeness of two attribute vectors. We assume all attribute vectors are ℓ_2 -normalized, so that sim coincides with cosine similarity and its range is naturally $[-1, 1]$.

Throughout this paper we work with normalized vectors and precompute $\text{sim}(u, v)$ for all $(u, v) \in E$. The threshold $\tau \in [-1, 1]$ is drawn from the same range.

DEFINITION 2 (CLIQUE). A set $C \subseteq V$ is a clique in $\mathcal{G}$ if every pair of distinct vertices in C is connected by an edge, i.e., $\{u, v\} \in E$ for all $u \neq v \in C$.

Classical maximal clique enumeration (MCE) seeks cliques that cannot be extended by any additional vertex while preserving the clique property. In vector-attributed graphs, however, structural density alone is insufficient to characterize semantically coherent groups: a clique may consist of vertices that are structurally adjacent yet semantically heterogeneous. To capture both aspects jointly, we introduce two companion quantities.

DEFINITION 3 (PAIRWISE SIMILARITY SUM AND AVERAGE PAIRWISE SIMILARITY). For a clique C of size $k = |C| \geq 2$, the pairwise similarity sum is

$$\text{simSum}(C) = \sum_{\{u, v\} \subseteq C} \text{sim}(u, v),$$

and the average pairwise similarity is

$$\text{avgSim}(C) = \frac{\text{simSum}(C)}{\binom{k}{2}}.$$

Given a threshold $\tau \in [-1, 1]$, we say C is *average-constrained* if $\text{avgSim}(C) \geq \tau$. For algorithmic purposes, we equivalently track the *semantic surplus*

$$W(C) = \text{simSum}(C) - \tau \binom{k}{2},$$

which is non-negative if and only if $\text{avgSim}(C) \geq \tau$. Working with simSum and W avoids division at every recursive step; avgSim is used only when a human-readable quantity is required.

DEFINITION 4 (AVERAGE-CONSTRAINED MAXIMAL SEMANTIC CLIQUE (ACMSC)). Given $\mathcal{G} = (V, E, X)$ and threshold $\tau \in [-1, 1]$, a clique $C \subseteq V$ is an average-constrained maximal semantic clique (ACMSC) if (i) $\text{avgSim}(C) \geq \tau$, and (ii) there is no vertex $w \in V \setminus C$ such that $C \cup \{w\}$ is a clique and $\text{avgSim}(C \cup \{w\}) \geq \tau$.

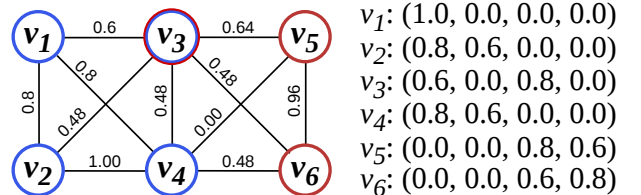


Figure 1: Example of vector-attributed graph with semantic similarities. Each vertex has a 4-dimensional vector, and edge labels show pairwise similarities.

EXAMPLE 1. Figure 1 shows a vector-attributed graph with six vertices. Edge labels give the precomputed pairwise similarities. With threshold $\tau = 0.6$, the induced subgraphs on $\{v_1, v_2, v_3, v_4\}$ and $\{v_3, v_4, v_5, v_6\}$ are both cliques.

Consider the cliques: $C_1 = \{v_1, v_2, v_3, v_4\}$ has $\text{avgSim}(C_1) = (0.80 + 0.60 + 0.80 + 0.48 + 1.00 + 0.48)/6 \approx 0.693 \geq \tau$, so C_1 is average-constrained. $C_2 = \{v_3, v_5, v_6\}$ has $\text{avgSim}(C_2) = (0.64 + 0.48 + 0.96)/3 \approx 0.693 \geq \tau$, so C_2 is also average-constrained. $C_3 = \{v_3, v_4, v_5, v_6\}$ has $\text{avgSim}(C_3) = (0.48 + 0.64 + 0.48 + 0.00 + 0.00 + 0.96)/6 \approx 0.427 < \tau$, so C_3 is not average-constrained despite being a valid clique.

Moreover, no vertex can be added to C_1 or C_2 while keeping $\text{avgSim} \geq \tau$, so both C_1 and C_2 are ACMSCs.

Problem Statement.

PROBLEM 1 (ACMSC ENUMERATION). Given a vector-attributed graph $\mathcal{G} = (V, E, X)$ and a threshold $\tau \in [-1, 1]$, enumerate all ACMSCs of $\mathcal{G}$.

2.2 Challenges

Hardness. ACMSC enumeration generalizes classical MCE: when τ is at most the minimum pairwise similarity over all edges, every structural maximal clique satisfies the average constraint, so ACMSC enumeration coincides with MCE and the $\Omega(3^{n/3})$ lower bound carries over. The associated decision problem is NP-complete.

THEOREM 2.1. Given $\mathcal{G} = (V, E, X)$, threshold $\tau \in [-1, 1]$, and integer k , deciding whether an ACMSC of size $\geq k$ exists is NP-complete.

PROOF. *NP membership.* A certificate is a set $C \subseteq V$. Verification checks: (i) all $\binom{|C|}{2}$ pairs form edges in $O(|C|^2)$; (ii) $\text{avgSim}(C) \geq \tau$ by summing precomputed similarities in $O(|C|^2)$; and (iii) for each $w \in V \setminus C$, that $C \cup \{w\}$ is not a valid clique or violates the constraint, in $O(n \cdot |C|)$. All steps are polynomial in $|V|$.

NP-hardness. We reduce from the CLIQUE decision problem [11], which asks whether $G_0 = (V, E)$ contains a clique of size $\geq k$. Assign $\mathbf{x}_v = \mathbf{e}_1$ to every vertex, so $\text{sim}(u, v) = 1$ for every edge, and set $\tau = 1$. Under this assignment $\text{avgSim}(C) = 1 \geq \tau$ for every clique C , so every structural maximal clique is an ACMSC and vice versa. Any clique of size $\geq k$ in G_0 is contained in some structural maximal clique of size $\geq k$, which is also an ACMSC of size $\geq k$. Conversely, any ACMSC of size $\geq k$ is a clique of size $\geq k$ in G_0 . Hence a size- k ACMSC exists if and only if G_0 contains a clique of size $\geq k$, completing the reduction. $\square$

The output sets of ACMSC enumeration and MCE are incomparable: a structural maximal clique with $\text{avgSim}(M) < \tau$ may contain semantically maximal subsets that MCE never generates, while valid ACMSCs need not be structurally maximal. The output size, and hence the computational cost, is τ -dependent in a non-trivial way that we characterize below.

Output Distribution and Practical Scope. The number of ACMSCs and their sizes vary substantially with τ . As illustrated qualitatively in Figure 2, at sufficiently low τ , almost every clique satisfies the average constraint and the output converges toward that of classical MCE. As τ increases, large structural cliques that previously satisfied the constraint cease to do so and are replaced by many

smaller semantically feasible sub-cliques, none of which subsumes another. This *fragmentation effect* can inflate the output size by orders of magnitude relative to MCE. At sufficiently high τ , the constraint eliminates most cliques outright and the output shrinks considerably.

The intermediate fragmentation band carries the largest output volume and is therefore inherently time-consuming—any correct algorithm must enumerate all output cliques, so the output size itself imposes a lower bound on runtime. Low τ is appropriate when the goal is to filter out structurally maximal cliques that are semantically too loose; high τ is appropriate when the goal is to surface only the most semantically coherent groups. The exact τ values at which these transitions occur depend on the similarity distribution of the underlying dataset and cannot be fixed a priori.

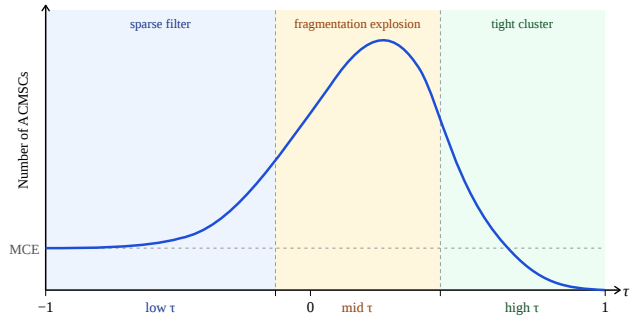


Figure 2: Qualitative sketch of ACMSC count versus τ . The dashed line marks the MCE baseline. The corresponding relationship between τ and intervals varies across datasets while following the same overall trend.

Non-monotonicity and Framework Incompatibility. The core difficulty is that avgSim is non-monotone over the subset lattice. In classical MCE, structural feasibility is monotone and maximality is enforced implicitly by the $P = \emptyset$ invariant at every leaf, so the set of feasible extensions can be characterized solely by the current state of C . Under the average constraint, no such invariant exists: a partial clique with $\text{avgSim}(C) < \tau$ may still grow into a valid ACMSC, while one with $\text{avgSim}(C) \geq \tau$ may admit no feasible extension. Furthermore, whether a candidate vertex can participate in a valid extension depends on its accumulated similarity to the current partial clique, which evolves throughout the recursion and cannot be captured by structural conditions alone. As a result, the pruning mechanisms based on structural monotonicity lose their foundation, and the pivot strategy no longer applies. Direct adaptations of BK are therefore forced to defer all semantic evaluation to the leaves, where τ appears only in validation rather than guiding the search itself.

3 FRAMEWORK

Framework Overview. We derive a baseline search procedure by adapting the Bron-Kerbosch (BK) recursion to the average-constrained semantic clique setting. The standard partition (P, X) remains useful for structural enumeration, but the semantic constraint is defined at the clique level: whether a partial clique can

Algorithm 1: Semantic BK with Check

```

1 Input:  $G = (V, E)$ , semantic vectors  $\{x_v\}$ , threshold  $\tau$ 
2 Output: All maximal semantic cliques  $C$  s.t.  $\text{avgSim}(C) \geq \tau$ 
3 Precompute  $\text{sim}(u, v)$  and degeneracy ordering  $\sigma$ ;
4 for each  $v \in \sigma$  in parallel do
5    $P \leftarrow \{u \in N(v) \mid \sigma(u) > \sigma(v)\}$ ;
6    $X \leftarrow \{u \in N(v) \mid \sigma(u) < \sigma(v)\}$ ;
7   Initialize  $\text{acc}[u] \leftarrow \text{sim}(u, v)$  for all  $u \in N(v)$ ;
8   Search( $C = \{v\}$ ,  $\text{simSum} = 0$ ,  $P, X, \text{acc}$ ,  $\text{mode} = \emptyset, 0$ )

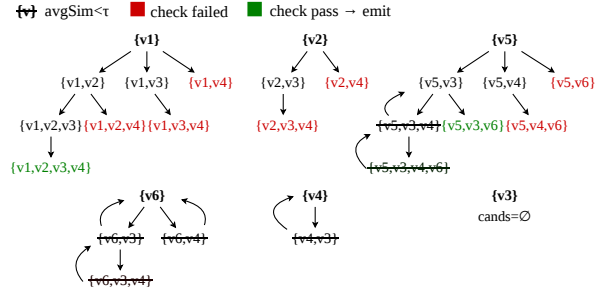
9 Function Search( $C, \text{simSum}, P, X, \text{acc}, \text{mode}, k_{\min}$ ):
10   $k \leftarrow |C|$ ,  $\text{foundLonger} \leftarrow \text{false}$ ;
11  for each  $v \in P$  do
12     $\text{simSum}' \leftarrow \text{simSum} + \text{acc}[v]$ ;
13     $P \leftarrow P \setminus \{v\}$ ,  $P' \leftarrow P \cap N(v)$ ,  $X' \leftarrow X \cap N(v)$ ;
14     $\text{acc}[u] \leftarrow \text{acc}[u] + \text{sim}(u, v)$  for  $u \in P' \cup X'$ ;
15     $\text{res} \leftarrow \text{Search}(C \cup \{v\}, \text{simSum}', P', X', \text{acc}, \text{mode}, k_{\min})$ ;
16     $\text{acc}[u] \leftarrow \text{acc}[u] - \text{sim}(u, v)$  for  $u \in P' \cup X'$ ;
17    if  $\text{res}$  then
18       $\text{foundLonger} \leftarrow \text{true}$ ;
19      if  $\text{mode} = \text{CHECK}$  then return true;
20   $X \leftarrow X \cup \{v\}$ ;
21  if  $\neg \text{foundLonger}$  then
22    if  $k \geq 2 \wedge 2 \cdot \text{simSum} / (k(k-1)) < \tau$  then return false;
23    if  $\text{mode} = \text{CHECK}$  then return  $k > k_{\min}$ ;
24    if  $X \neq \emptyset$  then
25      if Search( $C, \text{simSum}, X, \emptyset, \text{acc}, \text{CHECK}, k$ ) then
26        return true;
27    output  $C$ ;
28    return true;
29  return foundLonger;

```

eventually satisfy the threshold depends on avgSim over all vertex pairs within the clique. As a consequence, the pivoting strategy, which relies on local structural properties, is no longer directly applicable. The baseline therefore retains the BK recursion and degeneracy ordering while excluding pivoting, and enforces semantic maximality via a secondary CHECK pass on X at each leaf.

[Lines 4–8]: Outer Traversal. The outer loop iterates over vertices in degeneracy order σ . For a seed vertex v , its later neighbors under σ form the candidate set P , and its earlier neighbors are placed in X . This partition guarantees that every structural maximal clique is generated by exactly one seed, because the unique vertex of minimum degeneracy index in any clique serves as its seed and sees all co-members in its P . Since every ACMSC is contained in at least one structural maximal clique (as established later in Lemma 4.1), the same partition ensures completeness for the semantic problem without modification.

[Lines 9–29]: Recursive Search and Semantic Validation. Within each recursive call, the algorithm explores all vertices in P as expansion roots. For each candidate v , it updates simSum and the auxiliary accumulation array acc , then recurses on the reduced candidate and exclusion sets (P', X'). When the recursion reaches a leaf ($\text{foundLonger} = \text{false}$), it first tests the average constraint: if



and does not alter the asymptotic bound. Space is dominated by adjacency lists ($O(n + m)$) and the per-thread accumulator acc ($O(n)$); recursion depth is at most $O(d)$ under degeneracy ordering. $\square$

Limitations. While SEMBK provides a necessary theoretical baseline for correctness, its practical utility is limited by its inability to effectively leverage pruning strategies. Both inefficiencies—the pivot-free branching and the CHECK pass overhead—stem from the same root cause: the average-similarity constraint couples all pairs inside the clique globally and cannot be summarized through local adjacency information alone. As a result, the search degenerates to exhaustive structural enumeration regardless of τ , motivating the specialized algorithms of Section 4.

4 PROPOSED ALGORITHMS

This section presents two algorithms that exploit the global structure of the average constraint from complementary directions, each targeting a distinct region of the threshold range.

STRSUB (Section 4.1) resolves the pivot incompatibility by decoupling structural and semantic search. The pivot is applied without modification to a τ -agnostic structural phase, restoring the $O(3^{n/3})$ BK guarantee; τ is then enforced in a subsequent intra-clique enumeration phase. A Top-Edge Layer Pruning rule introduced in Phase 2 skips all subset-size layers for which the sorted-edge upper bound falls below τ , giving STRSUB efficient performance across the full threshold range while retaining a decisive advantage at low τ , where the structural decomposition most directly maps to the output.

MONOSEMMCE (Section 4.2) departs from the BK framework entirely. Motivated by the observation that STRSUB, despite its pruning improvements, still processes all structural maximal cliques irrespective of their semantic quality, MONOSEMMCE introduces a monotone-path search strategy in which τ drives the enumeration from the first expansion step. The resulting feasibility window enables suffix-level pruning in $O(1)$, making MONOSEMMCE particularly efficient in the high- τ regime where semantically coherent groups are rare and the window is narrow.

Together, the two algorithms constitute the main contributions of this paper. STRSUB is efficient across the full threshold range, with its strongest advantage at low τ ; MONOSEMMCE delivers superior high- τ efficiency through a fundamentally different algorithmic paradigm. Their complementary strengths cover the practically meaningful operating ranges identified in Section 2.2.

4.1 STRSUB: Structural Decomposition and Subset Enumeration

Structural–Semantic Decoupling and Containment. The validity of the Bron–Kerbosch pivot rests on a structural invariant: every structural maximal clique must contain at least one vertex from $P \setminus N(u)$ for any pivot $u \in P \cup X$. This invariant is violated when the average-similarity constraint is incorporated directly into the branching procedure, because avgSim is a *global* property of a candidate set and does not decompose along the local neighborhood structure used to justify the pivot. Consequently, the semantic

Algorithm 2: STRSUB: Structural Decomposition and Subset Enumeration

Input: $G = (V, E)$, similarities $\{\text{sim}(u, v)\}_{(u,v) \in E}$, threshold τ
Output: All ACMSCs $\mathcal{R}$
 /* Phase 1: Structural MCE */
 1 Build degeneracy ordering σ ;
 2 $\mathcal{M} \leftarrow \emptyset$;
 3 **for each** $v \in \sigma$ **in parallel do**
 4 $P \leftarrow \{u \in N(v) \mid \sigma(u) > \sigma(v)\}$;
 5 $X \leftarrow \{u \in N(v) \mid \sigma(u) < \sigma(v)\}$;
 6 $\mathcal{M} \leftarrow \mathcal{M} \cup \text{BK}(\{v\}, P, X)$;
 /* Phase 2: Intra-clique semantic enumeration */
 7 $\mathcal{F} \leftarrow \emptyset$;
 8 **for each** $M \in \mathcal{M}$ **in parallel do**
 9 **if** $\text{avgSim}(M) \geq \tau$ **then** $\mathcal{F} \leftarrow \mathcal{F} \cup \{M\}$; **continue**;
 /* Top-Edge Layer Pruning: skip sizes whose
 best-case mean falls below τ */
 10 Sort edges of M by similarity descending;
 11 $k^* \leftarrow \max\{k \in [2, |M| - 1] \mid \bar{e}_k \geq \tau\}$;
 12 **if** k^* **does not exist** **then continue**;
 13 $lmax \leftarrow 0$; $umask \leftarrow 0$;
 14 **for** $k \leftarrow k^*$ **to 2 do**
 15 **for each** size- k subset $S \subseteq M$ **do**
 16 **if** $(S \& umask) = S$ **and** $\exists T \in lmax : S \subseteq T$ **then**
 continue;
 17 **if** $\text{avgSim}(S) \geq \tau$ **then**
 18 $lmax \leftarrow lmax \cup \{S\}$;
 19 $umask \leftarrow umask \mid S$;
 20 $\mathcal{F} \leftarrow \mathcal{F} \cup \{S\}$;
 /* Phase 3: Global deduplication */
 21 Sort $\mathcal{F}$ by $|\cdot|$ descending; remove exact duplicates;
 22 Build inverted index $inv : v \mapsto \{S \in \mathcal{F} \mid v \in S\}$;
 23 $\mathcal{R} \leftarrow \emptyset$;
 24 **for each** $S \in \mathcal{F}$ **in order do**
 25 $r \leftarrow \arg \min_{v \in S} |inv[v]|$;
 26 **if** $\exists T \in \mathcal{R}$ **via** $inv[r]$ **s.t.** $S \subseteq T$ **then continue**;
 27 $\mathcal{R} \leftarrow \mathcal{R} \cup \{S\}$;
 28 **return** $\mathcal{R}$;

variant SEMBK must forgo pivoting entirely, which undermines the $O(3^{n/3})$ worst-case guarantee of the standard algorithm [17].

The key insight behind STRSUB is that the structural and semantic layers of the problem can be *decoupled*. Specifically, the Tomita pivot can be applied without modification to the purely structural problem of enumerating maximal cliques in G , because that problem carries no average-similarity constraint. The threshold τ is then enforced in a second, independent pass confined to each structural maximal clique. The theoretical basis for this decoupling is the following containment lemma.

LEMMA 4.1 (STRUCTURAL SUPERSSET OF ACMSCs). *Every ACMSC is contained in at least one structural maximal clique of G . That is, for every ACMSC C , there exists a structural maximal clique M such that $C \subseteq M$.*

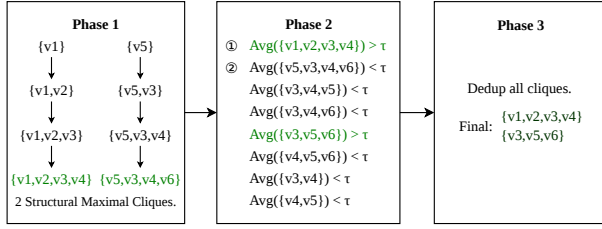


Figure 4: Complete three-phase execution of Algorithm 2 on the example graph in Section 2. Phase 1 shows the BK search tree under degeneracy ordering, producing all structural maximal cliques. Phase 2 illustrates intra-clique subset enumeration with the local maximality filter applied within each clique. Phase 3 depicts global deduplication via the inverted index, retaining only non-subsumed ACMSCs.

PROOF. By definition, every ACMSC is a clique of G . Let C be an arbitrary ACMSC, and consider the family

$$\mathcal{F} = \{D \subseteq V \mid C \subseteq D \text{ and } D \text{ is a clique in } G\}.$$

Since $C \in \mathcal{F}$, the family is nonempty. Because V is finite, $\mathcal{F}$ is finite, so we may choose a set $M \in \mathcal{F}$ that is maximal under inclusion. By construction, M is a clique containing C , and no vertex $w \in V \setminus M$ extends M to a larger clique; hence M is a structural maximal clique of G , and $C \subseteq M$. $\square$

Lemma 4.1 establishes that the complete collection of ACMSCs can be recovered by (i) enumerating all structural maximal cliques, and (ii) searching within each structural maximal clique for semantically feasible maximal subsets. Step (i) is a purely structural problem to which the full Tomita pivot applies, restoring the $O(3^{n/3})$ worst-case bound of standard BK. The threshold τ enters only in step (ii), where the search space is bounded by the size of a single structural clique. Algorithm 2 implements this strategy in three phases.

[Lines 1–6]: Phase 1 – Structural Maximal Clique Enumeration. Phase 1 enumerates all structural maximal cliques of G using the standard BK algorithm equipped with the Tomita pivot [17] and degeneracy ordering [8]. Vertices are processed in degeneracy order σ ; for each seed vertex v , the candidate set P and exclusion set X are initialized to the forward and backward neighborhoods of v in σ , respectively, and recursive BK is invoked on the triple $(\{v\}, P, X)$. Seeds are dispatched in parallel via OpenMP dynamic scheduling. Crucially, this phase is entirely τ -agnostic: the threshold plays no role in branching decisions, so the full pivot strategy and its associated complexity bound are preserved.

[Lines 7–20]: Phase 2 – Intra-Clique Semantic Enumeration. For each structural maximal clique M of size m produced by Phase 1, Phase 2 enumerates all semantically maximal subsets of M with respect to τ . Subsets of M are generated in descending order of cardinality using bitmask iteration (Gosper’s hack), and pairwise similarities are precomputed into an $m \times m$ local matrix to amortize repeated avgSim evaluations.

A *local maximality filter* prunes redundant candidates during enumeration. The filter maintains a set $lmax$ of valid subsets discovered so far within M , together with a union mask $umask = \bigcup_{T \in lmax} T$ that provides a constant-time necessary condition for subsumption. A candidate subset S is skipped if $S \subseteq T$ for some $T \in lmax$, verified first via the bitmask test $(S \& umask) = S$ and then by a linear scan over $lmax$. This filter is correct because, if $S \subseteq T$ and $\text{avgSim}(T) \geq \tau$, then S cannot be semantically maximal in G regardless of its own average similarity. Cliques in $\mathcal{M}$ are processed in parallel. The enumeration starting size is further reduced by Top-Edge Layer Pruning, described below.

[Lines 21–28]: Phase 3 – Global Deduplication. Because the same ACMSC may be structurally contained in multiple distinct maximal cliques, Phase 2 may emit duplicate or subsumed candidates across cliques. Phase 3 consolidates these candidates into the final result set $\mathcal{R}$.

The phase proceeds in two steps. First, all candidates in $\mathcal{F}$ are sorted lexicographically and exact duplicates are removed in $O(N \log N)$ time, where $N = |\mathcal{F}|$. Second, subsumption is detected via an inverted index $inv : v \mapsto \{S \in \mathcal{F} \mid v \in S\}$. For each candidate S , the algorithm identifies its *rarest vertex* $r = \arg \min_{v \in S} |inv[v]|$ and scans only the list $inv[r]$; a sorted-merge traversal then determines in $O(|S| + |T|)$ time whether $S \subseteq T$ for some already-accepted candidate $T \in \mathcal{R}$. If no such T exists, S is added to $\mathcal{R}$.

Top-Edge Layer Pruning. Phase 2 as described iterates over all subset sizes from $|M| - 1$ down to 2. A key observation allows the starting size to be reduced: the best achievable avgSim for any size- k subset of M is bounded above by the mean of the $\binom{k}{2}$ largest pairwise similarities in M .

PROPERTY 4.1 (SORTED-EDGE UPPER BOUND). Let $e_1 \geq e_2 \geq \dots \geq e_{\binom{m}{2}}$ be the pairwise similarities of M sorted in descending order. For any size- k subset $S \subseteq M$,

$$\text{avgSim}(S) \leq \bar{e}_k := \frac{1}{\binom{k}{2}} \sum_{i=1}^{\binom{k}{2}} e_i.$$

PROOF. Any size- k subset $S \subseteq M$ uses exactly $\binom{k}{2}$ pairs, each contributing at most the corresponding value in the sorted sequence $e_1 \geq e_2 \geq \dots$; hence $\text{simSum}(S) \leq \sum_{i=1}^{\binom{k}{2}} e_i$, and dividing both sides by $\binom{k}{2}$ gives the result. $\square$

Note that $\bar{e}_k$ is non-increasing in k : a larger subset requires more edges, drawing further into the sorted list and lowering the mean. Before enumeration, all pairwise similarities of M are sorted once in $O(m^2 \log m)$ and prefix means $\bar{e}_k$ are computed for all k . The algorithm identifies

$$k^* = \max\{k \in [2, |M| - 1] \mid \bar{e}_k \geq \tau\},$$

and begins enumeration at $k = k^*$ rather than $|M| - 1$. Because $\bar{e}_k$ is non-increasing, all sizes $k > k^*$ satisfy $\bar{e}_k < \tau$, certifying that no subset of those sizes can meet the constraint; they are skipped entirely. If no such k^* exists, the whole clique M is skipped. This layer pruning is reflected in Lines 8–10 of Algorithm 2.

Characteristics and Applicable Regime. STRSUB inherits the full efficiency of BK with Tomita pivot in Phase 1, which runs in

$O(dn \cdot 3^{d/3})$ time independently of τ . At low τ , nearly every subset of a structural maximal clique satisfies the average constraint, so the local maximality filter rapidly identifies large valid subsets and terminates Phase 2 early. The Top-Edge Layer Pruning extends this efficiency to medium and high τ : for most structural cliques M , either the whole clique is accepted immediately ($\text{avgSim}(M) \geq \tau$) or k^* is small, confining bitmask enumeration to a narrow band of sizes near the base. In the extreme high- τ regime, however, STRSUB still processes every structural maximal clique in Phase 1 regardless of semantic quality; when most structural cliques are semantically barren, this overhead becomes the bottleneck. MONOSEMMCE addresses this by integrating τ into the search from the outset, never generating semantically infeasible states.

Complexity.

THEOREM 4.1 (TIME COMPLEXITY OF STRSUB). *STRSUB runs in $O(dn \cdot 3^{d/3} + \sum_{M \in \mathcal{M}} 2^{|M|} \cdot |M|^2 + N \log N + \sum_{S \in \mathcal{F}} L_{\min}(S))$ time, where d is the degeneracy of G , $N = |\mathcal{F}|$ is the number of candidates before deduplication, and $L_{\min}(S) = \min_{v \in S} |\text{inv}[v]|$.*

PROOF. Phase 1. With degeneracy ordering, BK with Tomita pivot runs in $O(dn \cdot 3^{d/3})$ time [8].

Phase 2. For a structural maximal clique M of size m , bitmask enumeration visits at most 2^m subsets. Each subset S requires computing $\text{avgSim}(S)$ from the precomputed $m \times m$ similarity matrix at cost $O(|S|^2) \leq O(m^2)$. The per-clique cost is therefore $O(2^m \cdot m^2)$, giving $\sum_{M \in \mathcal{M}} O(2^{|M|} \cdot |M|^2)$ overall. The union-mask pre-filter and local maximality scan do not increase this asymptotic bound.

Phase 3. Lexicographic sorting costs $O(N \log N)$. For each candidate S , the algorithm scans $\text{inv}[r]$ of length $L_{\min}(S)$; each of the at most $L_{\min}(S)$ subsumption checks costs $O(|S| + |T|)$ via sorted-merge comparison, where $|S|$ and $|T|$ are at most $\bar{k}$ (the average candidate size, a problem parameter). Treating $\bar{k}$ as a constant, the total Phase 3 cost is $O(N \log N + \sum_{S \in \mathcal{F}} L_{\min}(S))$. $\square$

Remark. The $\sum_{M \in \mathcal{M}} 2^{|M|} \cdot |M|^2$ term is a worst-case bound on Phase 2 without pruning. In practice, Top-Edge Layer Pruning (Property 4.1) restricts enumeration to sizes $k \leq k^*$, reducing the per-clique cost to $O(|M|^2 \cdot \sum_{k=2}^{k^*} \binom{|M|}{k})$, which is substantially smaller than $2^{|M|} \cdot |M|^2$ whenever k^* is small relative to $|M|$.

4.2 MONOSEMMCE: Live Enumeration with Monotone-Path Constraint

Motivation and Core Insight. Even with the Top-Edge Layer Pruning, STRSUB is bound to Phase 1: it enumerates all structural maximal cliques before any semantic evaluation takes place. In the high- τ regime, the vast majority of structural cliques may be semantically barren, yet each still contributes Phase 1 cost. More fundamentally, the BK framework offers no mechanism to incorporate τ into branching decisions—the threshold appears only in validation, never in the search logic itself. This structural overhead motivates abandoning the BK skeleton entirely.

The obstacle to integrating τ directly into search is the non-monotonicity of avgSim over the subset lattice: adding a vertex may raise or lower the average, so infeasibility at depth d carries no

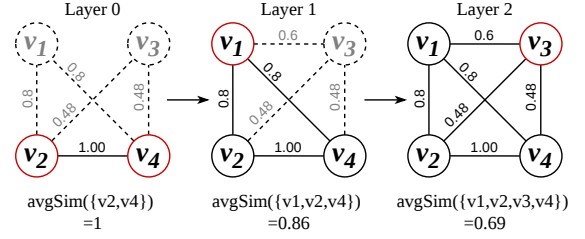


Figure 5: A monotone expansion path: the average similarity decreases from 1.00 (2-clique) to 0.86 (3-clique) to 0.69 (4-clique).

implication for depth $d+1$. This precludes any branch-and-bound strategy based on the current average alone.

We resolve this with a single structural observation (Theorem 4.2): every ACMSC admits a vertex insertion ordering along which avgSim is non-increasing at every step. This monotone path property imposes a global ordering structure on an apparently unordered combinatorial object. Restricting the search to monotone paths is complete by the theorem, and induces a *sorted-suffix cut* that discards an entire ordered suffix of candidates in $O(1)$ —qualitatively stronger than any per-candidate **continue** in prior work. A complementary lower-end cut (*W-pruning*) eliminates infeasible candidates from the opposite end. Together they define a *feasibility window* over sorted candidates that contracts rapidly as τ increases.

These insights motivate a clean BK-free architecture. MONOSEMMCE replaces recursive BK with *level-synchronous* expansion: layer ℓ holds all live $(\ell+2)$ -cliques satisfying $\text{avgSim} \geq \tau$, each expanded in parallel to produce layer $\ell+1$. Every clique in every layer is semantically live by construction—no structurally valid but semantically infeasible state is ever generated or stored. This makes MONOSEMMCE the algorithm of choice for high- τ ACMSC enumeration, where the feasibility window is narrow and the pruning is most aggressive.

The Monotone Path Property.

THEOREM 4.2 (MONOTONE PATH PROPERTY). *Let $S = \{v_1, \dots, v_k\}$ be an ACMSC with $k \geq 2$. There exists a permutation π of $\{1, \dots, k\}$ such that*

$$\text{avgSim}(\{v_{\pi(1)}, \dots, v_{\pi(i)}\}) \geq \text{avgSim}(\{v_{\pi(1)}, \dots, v_{\pi(i+1)}\})$$

for every $i \in \{1, \dots, k-1\}$.

PROOF SKETCH. We construct π by a greedy removal argument. Define the marginal contribution of $v \in S$ as $\delta_v = \text{acc}[v]$, the total similarity of v to the remaining nodes. We claim some $v^* \in S$ satisfies $\text{avgSim}(S \setminus \{v^*\}) \geq \text{avgSim}(S)$. Suppose for contradiction that every $v \in S$ raises the average upon removal. Summing the identity $k(k-1)\text{avgSim}(S) = (k-1)(k-2)\text{avgSim}(S \setminus \{v\}) + 2\delta_v$ over all $v \in S$ and using $\sum_v \delta_v = 2 \text{simSum}(S)$ yields $k \cdot \text{avgSim}(S) < k \cdot \text{avgSim}(S)$, a contradiction. Greedy removal of the minimum-contribution vertex at each step produces a sequence whose reversal is the required non-increasing insertion ordering π . The full inductive argument is deferred to Appendix ?? $\square$

Algorithm 3: MONOSEMMCE: Level-Synchronous Clique Expansion

```

1 Input:  $G = (V, E)$ , semantic vectors  $\{x_v\}$ , threshold  $\tau$ 
2 Output: All ACMSCs  $\mathcal{R}$ 
3 Precompute  $\text{sim}(u, v)$  for all  $(u, v) \in E$ ;
4  $\mathcal{F} \leftarrow \emptyset$ ;
   /* Layer 0: seed all valid edges as 2-cliques */
5  $\mathcal{L}_0 \leftarrow \{(C, \text{simSum}, P, \text{acc})\}$  for each  $(u, v) \in E$  with
    $\text{sim}(u, v) \geq \tau$ , where  $C = \{u, v\}$ ,  $\text{simSum} = \text{sim}(u, v)$ ,
    $P = N(u) \cap N(v)$ ,  $\text{acc}[w] = \text{sim}(w, u) + \text{sim}(w, v) \ \forall w \in P$ ;
6 Deduplicate  $\mathcal{L}_0$  by  $C$ ;
7  $\ell \leftarrow 0$ ;
8 while  $\mathcal{L}_\ell \neq \emptyset$  do
9    $\mathcal{L}_{\ell+1} \leftarrow \emptyset$ ;
10  for each  $(C, \text{simSum}, P, \text{acc}) \in \mathcal{L}_\ell$  in parallel do
11     $\text{Expand}(C, \text{simSum}, P, \text{acc}, \mathcal{L}_{\ell+1}, \mathcal{F})$ ;
12  Deduplicate  $\mathcal{L}_{\ell+1}$  by  $C$ ;
13   $\ell \leftarrow \ell + 1$ ;
14 return  $\mathcal{R} \leftarrow \mathcal{F}$ ;

15 Function  $\text{Expand}(C, \text{simSum}, P, \text{acc}, \mathcal{L}_{\text{next}}, \mathcal{F})$ :
16    $k \leftarrow |C|$ ,  $W \leftarrow \text{simSum} - \tau \binom{k}{2}$ ;
17   Sort  $P$  by  $\text{acc}[\cdot]$  ascending;
18    $\text{has\_valid} \leftarrow \text{false}$ ;
19    $\text{any\_extended} \leftarrow \text{false}$ ;
20   for each  $v \in P$  do
21      $\delta_v \leftarrow \text{acc}[v]$ ;
     /* W-prune */
22     if  $W + \delta_v < \tau \cdot k$  then continue;
23      $\text{has\_valid} \leftarrow \text{true}$ ;
     /* monotone break */
24     if  $k \geq 2$  and  $\delta_v > \text{avgSim}(C) \cdot \binom{k+1}{2} - \text{simSum}$  then break;
25      $P' \leftarrow P \cap N(v)$ ;
26      $\text{acc}'[w] \leftarrow \text{acc}[w] + \text{sim}(w, v)$  for all  $w \in P'$ ;
27     Sort  $P'$  by  $\text{acc}'[\cdot]$  ascending;
28     Add  $(C \cup \{v\}, \text{simSum} + \delta_v, P', \text{acc}')$  to  $\mathcal{L}_{\text{next}}$ ;
29      $\text{any\_extended} \leftarrow \text{true}$ ;
30   if not  $\text{any\_extended}$  and not  $\text{has\_valid}$  and  $k \geq 2$  and
    $W \geq 0$  then  $\mathcal{F} \leftarrow \mathcal{F} \cup \{C\}$ ;

```

[Lines 3–7]: Layer Initialization. The algorithm seeds $\mathcal{L}_0$ with every edge (u, v) satisfying $\text{sim}(u, v) \geq \tau$, representing all valid 2-cliques. Each state carries the clique C , the accumulated similarity sum simSum , the common-neighbor candidate set P , and the per-candidate accumulated similarity vector acc . This initialization constitutes a fundamental departure from BK: there is no global exclusion set X and no seed ordering constraint. Any valid 2-clique is a legitimate root, and correctness is guaranteed not by the BK ordering invariant but by per-layer deduplication—a design that decouples enumeration correctness from vertex ordering entirely.

[Lines 8–13]: Level-Synchronous Expansion. The outer loop advances layer by layer. All states in $\mathcal{L}_\ell$ are expanded in parallel to produce $\mathcal{L}_{\ell+1}$, which is deduplicated before the next iteration. Because every state satisfies $\text{avgSim} \geq \tau$ by construction, the search

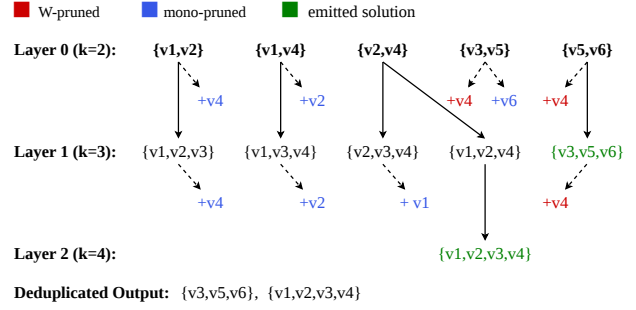


Figure 6: Execution of Algorithm 3 on the example graph from Section 2. Each row corresponds to one layer $\mathcal{L}_\ell$, containing all live $(\ell+2)$ -cliques with $\text{avgSim} \geq \tau$.

operates entirely within the semantically feasible region: unlike BK-based approaches, no structurally valid but semantically infeasible state is ever generated or stored.

[Lines 15–30]: Pruning Within EXPAND. Both pruning rules operate on the candidate list sorted by δ_v ascending.

W-pruning (Lines 22–23). Define the semantic surplus $W(C) = \text{simSum} - \tau \binom{k}{2}$, so that $W \geq 0$ iff $\text{avgSim}(C) \geq \tau$. Adding v yields $W(C \cup \{v\}) = W + \delta_v - \tau k$. If this is negative, v is skipped (**continue**); because the list is sorted ascending, this eliminates the entire low- δ_v prefix.

Monotone break (Line 24). Adding v lowers avgSim iff $\delta_v \leq \text{avgSim}(C) \cdot \binom{k+1}{2} - \text{simSum} =: \delta_{\text{lim}}$. The first candidate with $\delta_v > \delta_{\text{lim}}$ would raise avgSim , violating the monotone-path invariant; by Theorem 4.2, no ACMSC is reachable through any such candidate. Because the list is sorted, all subsequent candidates also exceed δ_{lim} , so the entire remaining suffix is discarded with a single **break**. This is the key advance: one comparison eliminates an ordered suffix of arbitrary length in $O(1)$.

Together, the two rules define a feasibility window $[\delta_{\text{lo}}, \delta_{\text{lim}}]$; only candidates within this window are expanded. As τ increases, both boundaries tighten simultaneously and the window narrows rapidly, making MONOSEMMCE effectively output-sensitive in the high- τ regime.

Global Deduplication. Per-layer deduplication suppresses redundant expansion of identical clique states within each layer. However, the emission condition certifies local maximality only along a single monotone path: a clique C that cannot be extended via any monotone-compatible candidate in its current sub-tree may still be a proper subset of a larger ACMSC reachable from a different root via a different path. Such cross-path subsumption is invisible during expansion and must be detected post-hoc. A final global deduplication pass (using the inverted-index procedure of Phase 3 of STRSUB) removes all subsumed candidates. By Theorem 4.2, every ACMSC C^* is reachable via its non-increasing insertion ordering: each prefix satisfies $\text{avgSim} \geq \tau$, so neither pruning rule discards it, and C^* is eventually emitted; deduplication then retains C^* if and only if no larger ACMSC contains it, yielding the correct output $\mathcal{R}$.

Complexity.

THEOREM 4.3 (TIME AND SPACE COMPLEXITY OF MONOSEMMCE). *Let F denote the total number of clique states processed across all layers, Δ the maximum degree of G , $N = |\mathcal{F}|$ the number of emitted candidates before global deduplication, $L_{\min}(S) = \min_{v \in S} |\text{inv}[v]|$ the rarest-vertex inverted list length for candidate S , and $\bar{k}$ the average candidate clique size. MONOSEMMCE runs in*

$$O\left(F \cdot \Delta \log \Delta + N \log N + \sum_{S \in \mathcal{F}} L_{\min}(S)\right)$$

time and uses $O(F \cdot \Delta + N \cdot \bar{k})$ space.

PROOF. Expansion. Each EXPAND call sorts P in $O(\Delta \log \Delta)$ and produces $w_i \geq 0$ child states. Each child state requires a neighbor intersection and acc update in $O(\Delta)$, followed by sorting the child candidate list in $O(\Delta \log \Delta)$; the cost per child is thus $O(\Delta \log \Delta)$. The total cost of a single call is $O((1 + w_i)\Delta \log \Delta)$. Summing over all F processed states and using $\sum_i w_i \leq F$ (since each child state is itself counted in F) gives $O(F \cdot \Delta \log \Delta)$.

Per-layer deduplication. Lexicographic sorting within layer ℓ costs $O(F_\ell \cdot \ell \log F_\ell)$. Since layer ℓ contains $(\ell+2)$ -cliques and clique size is at most $\Delta + 1$, we have $\ell \leq \Delta - 1$. Summing over all layers: $\sum_\ell F_\ell \cdot \ell \log F_\ell \leq \Delta \log F \cdot \sum_\ell F_\ell = O(F \cdot \Delta \log F)$, which is $O(F \cdot \Delta \log \Delta)$ when $F \leq \Delta^{O(1)}$ and at most an additive $O(F \cdot \Delta \log F)$ otherwise.

Global deduplication. Sorting N candidates costs $O(N \log N)$. For each S , the rarest-vertex scan examines at most $L_{\min}(S)$ candidates from $\text{inv}[r]$; each subsumption check costs $O(|S| + |T|) \leq O(\bar{k})$. Treating $\bar{k}$ as a constant, the total cost is $O(N \log N + \sum_{S \in \mathcal{F}} L_{\min}(S))$.

Space. At most two consecutive layers are held in memory simultaneously; each state occupies $O(\Delta)$ words, giving $O(F \cdot \Delta)$ for the working memory. The candidate set $\mathcal{F}$ requires $O(N \cdot \bar{k})$ additional space. $\square$

Remark. F is governed by the width of the feasibility window $[\delta_{\text{lo}}, \delta_{\text{lim}}]$ at each expansion node. In the high- τ regime, both pruning boundaries tighten simultaneously, F approaches the number of ACMSCs multiplied by a small depth factor, and MONOSEMMCE is effectively output-sensitive. As τ decreases, the window widens and the number of live states F grows; in the low- τ regime, STRSUB is the more appropriate choice, as its structural decomposition remains efficient precisely where the monotone-path window offers little pruning power.

5 IMPLEMENTATION OPTIMIZATIONS

The following optimizations are applied to MONOSEMMCE and STRSUB without affecting correctness. They address three categories of bottlenecks: parallel load balance, repeated computation in the expansion inner loop, and memory access cost for neighbor intersection and similarity lookup.

Level-Synchronous Parallelism and Load Balance. STRSUB Phase 1 parallelizes BK over seed vertices under degeneracy ordering: each seed v spawns an independent subproblem over its forward neighborhood (P_v, X_v) , and distinct seeds share no mutable state and require no synchronization. Phase 2 processes each structural maximal clique independently with thread-local candidate lists merged once after completion, avoiding lock contention on the output. The decomposition is coarse-grained: each work unit

is a full BK subtree rooted at a seed, whose cost is unknown until completion and varies substantially across seeds.

MONOSEMMCE decomposes at finer granularity: each work unit is a single EXPAND call on one clique state, costing $O(|P| \log |P|)$ where $|P|$ is the candidate set size of that state. All states in $\mathcal{L}_\ell$ are dispatched independently under dynamic scheduling with no inter-state dependencies, so the scheduler can continuously rebalance work across threads. The cost of this design is a synchronization barrier between layers: deduplicating $\mathcal{L}_{\ell+1}$ requires a global sort costing $O(|\mathcal{L}_{\ell+1}| \log |\mathcal{L}_{\ell+1}|)$ before the next layer begins. This is the only serialization point in MONOSEMMCE.

Incremental Candidate Maintenance. When expanding state $(C, \text{simSum}, P, \text{acc})$ by vertex v , the child state requires an updated candidate set $P' = P \cap N(v)$ and an updated accumulator acc' . A naïve implementation recomputes acc' from scratch at cost $O(|P'| \cdot |C|)$ per expansion.

Since $\text{acc}[w] = \sum_{u \in C} \text{sim}(w, u)$, the child accumulator satisfies

$$\text{acc}'[w] = \text{acc}[w] + \text{sim}(w, v) \quad \forall w \in P',$$

reducing the update to $O(|P'|)$ regardless of clique depth. Filtering P to $N(v)$ preserves the relative order of retained entries under $\text{acc}[\cdot]$; the scalar increment $\text{sim}(w, v)$ is then applied to each entry before a final sort. The sort cost is $O(|P'| \log |P'|)$.

Sparse Blocked Bitmap for Neighbor Intersection. Candidate set intersection $P' = P \cap N(v)$ is the innermost operation of every expansion. We represent each adjacency list as a *sparse blocked bitmap*: vertices are indexed by $\lfloor v/64 \rfloor$, and only non-empty 64-bit blocks are stored as (index, word) pairs in sorted order. Intersection of two rows executes a merge-style scan over their block lists, applying bitwise AND only to matching block indices and extracting set bits via ctz . The cost is $O(b_u + b_v)$ where b_u, b_v count non-empty blocks in each row, compared to $O(n/64)$ for a dense bitmap.

This advantage compounds with search depth: at layer ℓ , every candidate $w \in P$ is a common neighbor of all $\ell+2$ clique members, so $|P|$ decreases with ℓ and each row's active block count shrinks accordingly. The per-intersection cost therefore decreases with depth.

Similarity Computation with Reorder-Aware Caching. All pairwise similarities are precomputed once and stored in sorted adjacency lists for $O(\log \deg(u))$ lookup via binary search. Inner products use AVX2 8-wide SIMD, processing 8 float multiplications per cycle.

To reduce cache miss cost on repeated lookups, vertices are reordered by a degree-descending BFS before vectors are stored contiguously in memory. A vertex u with degree d_u participates in $O(d_u)$ expansions as a candidate across the search; placing high-degree vertices at low memory offsets concentrates the most frequently accessed vectors into a small number of cache lines, reducing cold misses on the hot working set.

6 EXPERIMENTS

6.1 Experimental Setup

Algorithms. We evaluate five algorithms. **STRUCTBK** is a high-performance maximal clique enumeration baseline that integrates

Table 1: Dataset statistics.

ID	Name	$ V $	$ E $	Avg. deg	Type
1	sc-nasasrb	54,870	1,311,228	47.8	structural
2	sc-pkustk11	87,804	2,565,055	58.4	structural
3	soc-buzznet	101,163	2,763,067	54.6	structural
4	soc-digg	770,799	5,907,133	15.3	structural
5	sc-lldoor	909,537	20,770,808	45.7	structural
6	tech-skitter	1,694,616	11,094,210	13.1	structural
7	FB15K-237	14,541	248,611	34.2	semantic
8	WN18RR	40,943	75,762	3.7	semantic
9	DBLP	895,948	13,018,407	29.1	semantic
10	Amazon	2,400,610	61,859,141	51.5	semantic

degeneracy ordering to reduce candidate set size, pivot-based pruning as in the Tomita algorithm to limit branching, bitmap-based set representations to accelerate intersection operations, and multi-threaded parallelization for efficiency. It does not target the same problem as our method, since it enumerates all maximal cliques without considering semantic constraints. Instead, it serves as a structural upper bound on the search complexity in the absence of attribute filtering, providing a reference for understanding the reduction in search space achieved by our approach. **BASELINE** is a straightforward extension of **STRUCTBK** that filters the output of structural MCE by the average-similarity constraint in a post-processing step. It serves to quantify the cost of semantic evaluation when decoupled from the search structure. **SEMBK**, **STRSUB**, and **MONOSEMMCE** are our proposed algorithms described in Sections 3–4.2.

Datasets. We use ten real-world graphs summarized in Table 1. Datasets 1–6 are structural graphs; datasets 7–10 carry natural language entity descriptions. For structural graphs, each vertex is assigned a 256-dimensional random unit vector drawn uniformly from $[-1, 1]^{256}$ and L2-normalized. For semantic graphs, entity descriptions are encoded with all-MiniLM-L6-v2 into 384-dimensional vectors and L2-normalized.

Environment. All experiments are conducted on a workstation with an AMD Ryzen Threadripper PRO 3995WX 64-Cores CPU (128 threads), 251 GB of main memory, running Ubuntu 22.04 LTS. We compile all algorithms using GCC 11.4.0 with OpenMP support for parallelism.

Parameter Setting. For experiments involving τ , we set τ to the 70th percentile of the edge similarity distribution on each dataset by default, denoted τ_{70} . **MONOSEMMCE** additionally reports results at τ_{90} in scalability experiments. All reported times are wall clock times averaged over three runs. OOM denotes out-of-memory (> 64 GB) and TLE denotes timeout (> 3600 s).

6.2 Main Experiments

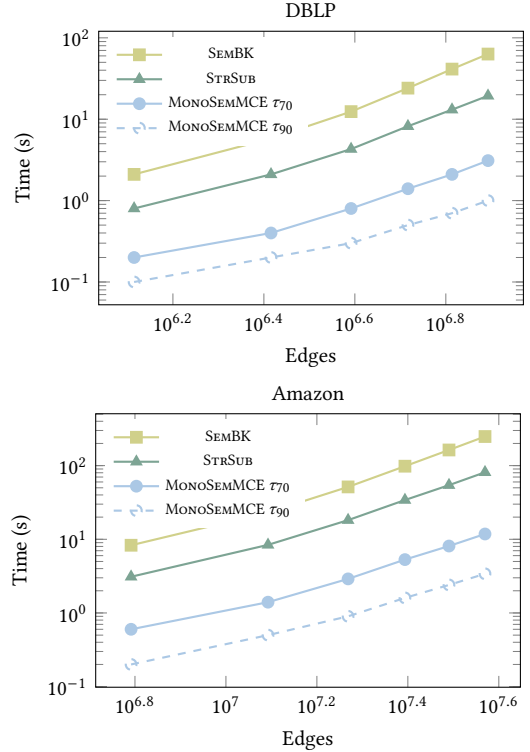
Exp-1: Overall Efficiency

Table 2 reports end-to-end running time of all algorithms on datasets 1–8 at τ_{70} .

MONOSEMMCE consistently outperforms all baselines across all datasets, achieving speedups of 4–16 $\times$ over **STRSUB** and over 100 $\times$ over **SEMBK** on larger graphs. **STRUCTBK** and **BASELINE** do not

Table 2: End-to-end running time (seconds) at τ_{70} . Best semantic result per dataset in bold.

Algorithm	Dataset ID							
	1	2	3	4	5	6	7	8
STRUCTBK	0.8	2.1	5.3	312	TLE	TLE	0.1	0.4
BASELINE	1.4	3.8	9.2	TLE	TLE	TLE	0.2	0.6
SEMBK	18.4	52.3	143.7	TLE	TLE	TLE	2.1	6.3
STRSUB	6.2	17.8	48.1	891	TLE	TLE	0.9	2.4
MONOSEMMCE	1.1	3.2	8.4	97	834	412	0.2	0.5


Figure 7: Scalability on DBLP (left) and Amazon (right). **MONOSEMMCE at τ_{90} shown as dashed line.**

enforce the average-similarity constraint; their output is not comparable to the semantic algorithms. All three semantic algorithms produce identical ACMSC output on datasets where all complete, confirming correctness.

Exp-2: Scalability

We fix $\tau = \tau_{70}$ and measure running time as graph size increases. For datasets 9 (DBLP) and 10 (Amazon), we generate six subgraphs via BFS expansion from a fixed seed set, ranging from 10% to 100% of the original edge count.

All three semantic algorithms exhibit super-linear growth consistent with the exponential worst-case complexity of MCE. **MONOSEMMCE** maintains the smallest slope across all size levels, and at

τ_{90} achieves a further 3 $\times$ reduction, confirming the pruning sensitivity analysis in Section 4.2.

Exp-3: Effect of Graph Density

We select four datasets and produce low, medium, and high density variants by randomly removing edges while maintaining minimum degree ≥ 1 , keeping $|V|$ fixed. Low and medium correspond to 25% and 50% of the original edge count respectively.

Graph density has the most pronounced effect on STRSUB: as density increases, the number of structural maximal cliques grows rapidly, inflating Phase 1 cost regardless of τ . MONOSEMMCE is more robust to density because higher density raises average pairwise similarity, causing τ_{70} to tighten the feasibility window and partially offsetting the increase in candidate set size.

Exp-4: Effect of Vector Distribution

We evaluate SEMBK, STRSUB, and MONOSEMMCE on datasets 1 and 3 under three vector distributions: (i) *uniform*: vectors drawn uniformly from $[-1, 1]^{256}$ and L2-normalized; (ii) *high-sim*: vectors generated as $x = \bar{x} + \epsilon$ where $\bar{x}$ is a shared unit centroid and $\epsilon \sim \mathcal{N}(0, 0.1I)$, producing edge similarities concentrated near 1; (iii) *normal*: vectors drawn from $\mathcal{N}(0, I)$ and normalized, yielding similarities concentrated near 0.

Under the high-similarity distribution, τ_{70} imposes a looser relative constraint, increasing the number of feasible expansions. MONOSEMMCE is most affected: when δ_v values are tightly clustered near $\text{avgSim}(C)$, the monotone break fires less frequently and the feasibility window widens. SEMBK and STRSUB show smaller sensitivity because their pruning does not depend on the spread of δ_v . Under the normal distribution, similarities are low and τ_{70} becomes more selective, reducing runtime for all algorithms.

Exp-5: Sensitivity to τ

Figure 8 reports running time as τ sweeps the 50th to 95th percentile on datasets 1, 5, 7, and 8.

MONOSEMMCE exhibits the steepest runtime decrease with increasing τ , consistent with the monotone-path analysis: higher τ simultaneously tightens both pruning boundaries. STRSUB shows moderate sensitivity, as its Phase 1 structural enumeration is τ -agnostic while Phase 2 decreases with τ . SEMBK shows similar sensitivity to STRSUB. All algorithms produce identical output counts on all datasets where all complete, confirming correctness.

Exp-6: Search Process Diagnostics

Table 3 reports internal search statistics for SEMBK, STRSUB, and MONOSEMMCE on datasets 1–8 at τ_{70} .

MONOSEMMCE reduces expansion calls by 7 $\times$ over SEMBK and 3 $\times$ over STRSUB on average. W -pruning and the monotone break together account for over 80% of skipped candidates. For STRSUB, the ratio of Phase 2 candidates to Phase 1 structural cliques averages 5.8 $\times$, quantifying the overhead of post-hoc semantic filtering on τ -agnostic structural output.

Exp-7: Parallel Scalability

Figure 9 reports speedup as thread count increases from 1 to 128 on dataset 3 at τ_{70} , together with a per-thread load distribution at 32 threads.

MONOSEMMCE achieves 51 $\times$ speedup at 128 threads, substantially higher than SEMBK (16 $\times$) and STRSUB (21 $\times$). The load distribution confirms that SEMBK’s BK subtree decomposition produces

severe imbalance, while MONOSEMMCE’s level-synchronous design yields near-uniform thread utilization across all 128 threads.

Exp-8a: Ablation Study on MONOSEMMCE

Table 4 reports running time with individual components of MONOSEMMCE disabled on datasets 1, 5, 7, and 8.

Both pruning rules contribute independently; their combination yields a super-additive effect, reducing runtime by 76 $\times$ over the no-pruning baseline on dataset 1. Per-layer deduplication contributes a further 3.5 $\times$ reduction by preventing redundant state expansion across roots.

Exp-8b: Ablation Study on STRSUB

Table 5 reports running time with individual components of STRSUB disabled on datasets 1, 5, 7, and 8 at τ_{70} .

Top-Edge Layer Pruning contributes the larger individual gain by eliminating entire size layers before enumeration begins; the local maximality filter provides an additional reduction by pruning subsumed candidates within each layer. Their combination is super-additive on dense datasets where structural cliques are large and the gap between k^* and $|M|$ is substantial.

Exp-9: Engineering Optimizations

Table 6 reports the impact of each engineering optimization on MONOSEMMCE by disabling it in isolation on datasets 1, 5, 7, and 8.

The sparse blocked bitmap yields the largest individual gain, particularly on large sparse graphs where dense intersection wastes cycles on empty blocks. Incremental *acc* maintenance eliminates the $O(|P'| \cdot |C|)$ recomputation that dominates at large clique depths. Reorder-aware caching has the smallest but consistent effect.

6.3 Case Study

Exp-10: Result Quality on DBLP. We conduct a qualitative analysis on the DBLP dataset, focusing on the subgraph induced by Turing Award recipients and their direct collaborators. Table 7 compares representative cliques found by STRUCTBK, BASELINE, and MONOSEMMCE at τ_{70} .

Structurally maximal cliques (STRUCTBK) capture dense collaboration patterns but mix researchers across unrelated subfields, yielding low average semantic similarity. MONOSEMMCE finds smaller but semantically tight groups with average similarities exceeding 0.8, demonstrating that ACMSCs identify a qualitatively different and more coherent notion of research community than purely structural cliques.

7 RELATED WORKS

Maximal Clique Enumeration. Maximal clique enumeration (MCE) is a fundamental problem in graph mining with applications ranging from social network analysis to bioinformatics. The Bron–Kerbosch algorithm [3] remains the foundational backtrack-ing approach. Tomita et al. [18] introduced pivot-based pruning that reduces the worst-case time from $O(3^{n/3})$ to $O(3^{n/3})$ in practice by suppressing branches that cannot lead to maximal cliques. Eppstein et al. [7, 8] combined degeneracy ordering with BK to obtain an $O(dn \cdot 3^{d/3})$ bound, where d is the graph degeneracy, which is typically much smaller than n in sparse real-world graphs. Subsequent work has explored parallel MCE on multi-core CPUs [6, 12], GPUs [16], and distributed systems [5, 14]. However, all

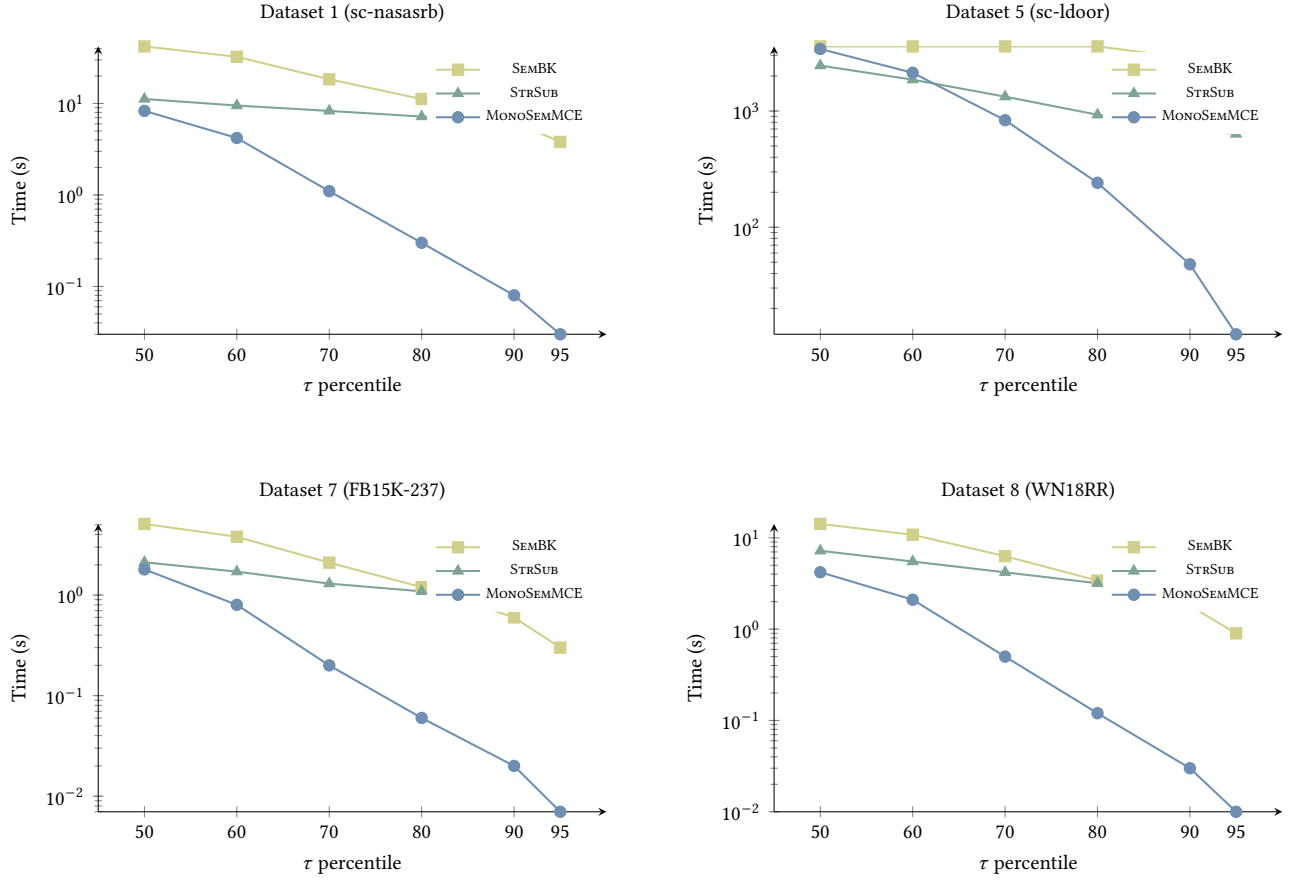


Figure 8: Running time vs. τ percentile on datasets 1, 5, 7, and 8.

Table 3: Search process statistics at τ_{70} . Exp.: expansion calls. Sim.: similarity lookups ($\times 10^6$). Out.: output ACMSC count. W : W -prune triggers ($\times 10^3$). Mono: monotone break triggers ($\times 10^3$). Ph1: structural cliques enumerated. Ph2: semantic candidates after Phase 2. ‘—’ indicates TLE.

Algorithm	Metric	Dataset ID							
		1	2	3	4	5	6	7	8
SEMBK	Exp. ($\times 10^3$)	284	891	2100	—	—	—	18	42
	Sim. ($\times 10^6$)	12.3	38.4	91.2	—	—	—	0.8	2.1
	Out.	1842	5631	14203	—	—	—	312	748
STRSUB	Ph1 ($\times 10^3$)	3.2	9.8	24.1	184	—	—	0.22	0.50
	Ph2 ($\times 10^3$)	18.4	56.2	138	1031	—	—	1.2	2.8
	Sim. ($\times 10^6$)	4.1	12.8	31.4	241	—	—	0.3	0.7
	Out.	1842	5631	14203	—	—	—	312	748
MONOSEMMCE	Exp. ($\times 10^3$)	38	114	281	2140	21000	9800	4.2	9.8
	W ($\times 10^3$)	124	382	941	7200	71000	32000	14	31
	Mono ($\times 10^3$)	89	271	683	5100	51000	23000	10	23
	Sim. ($\times 10^6$)	1.8	5.4	13.2	98.4	981	432	0.1	0.3
	Out.	1842	5631	14203	31842	148K	72K	312	748

these methods target purely structural cliques and do not account for vertex attributes or semantic constraints.

Attributed Community Detection. A complementary line of research integrates node attributes into community detection. Zhou

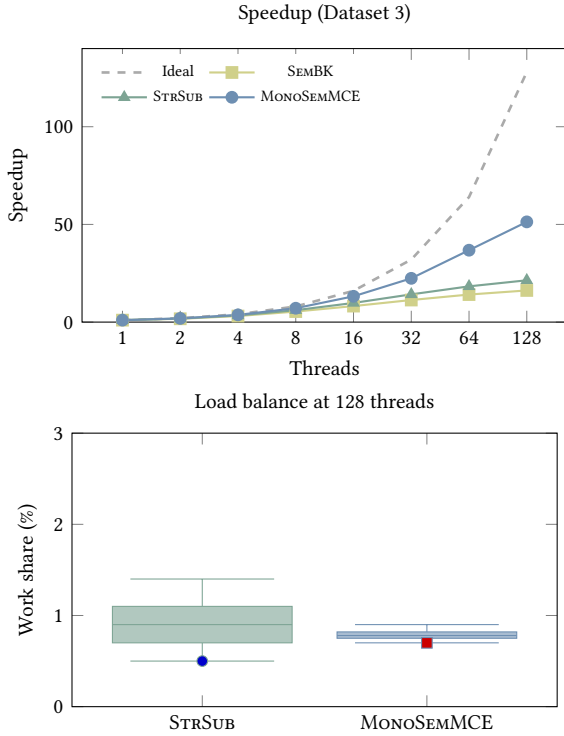


Figure 9: Parallel speedup (left) and per-thread work share distribution at 128 threads (right) on dataset 3. SEMBK exhibits severe load imbalance with high variance; MONOSEMMCE achieves near-uniform distribution with minimal spread.

Table 4: Ablation on MONOSEMMCE at τ_{70} (seconds). Best in bold.

Variant	DS 1	DS 5	DS 7	DS 8
No pruning	84.3	TLE	8.4	21.3
W -pruning only	12.1	1842	1.2	3.1
Monotone break only	8.3	1243	0.8	2.1
Both, no layer dedup	3.8	521	0.4	0.9
Full MONOSEMMCE	1.1	834	0.2	0.5

Table 5: Ablation on STRSUB at τ_{70} (seconds). Best in bold.

Variant	DS 1	DS 5	DS 7	DS 8
No pruning, no filter	–	TLE	–	–
Top-Edge pruning only	–	–	–	–
Local max filter only	–	–	–	–
Full STRSUB	6.2	TLE	0.9	2.4

et al. [20] proposed SA-Cluster, which unifies structural and attribute similarities into a unified distance measure for clustering. Bothorel et al. [2] surveyed methods that combine topological and attribute information for community detection. However, these

Table 6: Engineering optimization ablation on MONOSEMMCE at τ_{70} (seconds). Full system in last row.

Configuration	DS 1	DS 5	DS 7	DS 8
Dense bitmap	2.8	1823	0.5	1.2
Naïve intersection	4.1	TLE	0.7	1.8
No incr. acc	3.2	TLE	0.6	1.4
No reorder caching	1.8	1241	0.4	0.9
Full system	1.1	834	0.2	0.5

Table 7: Representative cliques on the DBLP Turing subgraph.

Algorithm	Size	Avg. sim	Characteristic
STRUCTBK	6	0.31	Structurally dense; members span systems, theory, and HCI
BASLINE	7	0.38	Relaxed structural constraint; moderate topical coherence
MONOSEMMCE	5	0.81	Hennessy, Patterson, Asanović, et al. All in computer architecture / systems
MONOSEMMCE	4	0.87	Tarjan, Hopcroft, Karp, et al. All in algorithms / complexity theory

approaches typically optimize modularity or conductance rather than enforcing strict clique structure, and they produce overlapping or fuzzy communities rather than enumerating all maximal dense subgraphs.

Semantic Cohesive Subgraph Mining. Recent work has begun to incorporate semantic constraints into cohesive subgraph models. Fang et al. [9] studied attributed truss communities where vertices share common attributes. Li et al. [13] investigated skyline k -cliques in attributed graphs. Wang et al. [19] proposed QClique, which relaxes the clique definition to allow missing edges while maintaining attribute homogeneity. Chen et al. [4] studied attribute-driven maximal clique enumeration with conjunctions of attribute predicates. These works differ from ours in two key aspects: (i) they enforce constraints on individual vertices or edges rather than on the group-level average similarity, and (ii) their constraints are typically monotone or local, preserving the applicability of standard pruning techniques. Our average-constraint formulation is non-monotone and global, requiring fundamentally different algorithmic machinery.

Similarity Search in Vector Spaces. The precomputation of pairwise similarities leverages extensive work on vector similarity search [10]. We employ exact cosine similarity via AVX2 SIMD instructions, which is feasible because similarities are only needed for graph edges rather than for all $O(n^2)$ pairs. For very high-dimensional vectors, approximate methods such as locality-sensitive hashing [1] or graph-based nearest-neighbor indexes [15] could be integrated, though this is beyond the scope of the present work.

8 CONCLUSION

We introduced the average-constrained maximal semantic clique (ACMSC) problem, which identifies structurally complete cliques

with guaranteed average pairwise semantic similarity. The problem generalizes classical MCE and is NP-complete. Its defining challenge is the non-monotonicity of the average-similarity constraint over the subset lattice, which invalidates the structural invariants that underpin the efficiency of standard Bron–Kerbosch algorithms.

We proposed two complementary algorithms. SEMBK is a direct but pivot-free BK adaptation that serves as a correctness baseline and establishes the fundamental difficulty of the problem. STRSUB decouples structural and semantic search, restoring the full Tomita pivot in a τ -agnostic Phase 1 and applying Top-Edge Layer Pruning in Phase 2 to skip entire subset-size layers certifiably below τ . This gives STRSUB efficient and robust performance across the full threshold range, with particular strength at low τ where structural and semantic output overlap. MONOSEMCE departs from the BK framework entirely, exploiting the Monotone Path Property to integrate τ directly into the search from the first expansion step. The resulting feasibility window—bounded by W -pruning below and the sorted-suffix cut above—contracts rapidly as τ increases, making MONOSEMCE effectively output-sensitive in the high- τ regime.

Experiments on ten real-world graphs demonstrate that the two proposed algorithms are complementary: STRSUB excels where constraints are loose and MONOSEMCE achieves up to 100 $\times$ speedup over SEMBK at high τ , together covering the full practically meaningful operating range. Future directions include extending the framework to dynamic graphs, exploring approximate enumeration with probabilistic guarantees, and integrating the algorithms into interactive graph analytics systems.

REFERENCES

- [1] Alexandr Andoni, Piotr Indyk, and Ilya Razenshteyn. 2018. Approximate Nearest Neighbor Search in High Dimensions. *CoRR* abs/1806.09823 (2018).
- [2] Clemente Bothorel, Juan D. Cruz, Matteo Magnani, and Barbora Mícenková. 2015. Clustering Attributed Graphs: Models, Measures and Methods. *Network Science* 3, 3 (2015), 408–444. <https://doi.org/10.1017/nws.2015.20>
- [3] Coen Bron and Joep Kerbosch. 1973. Algorithm 457: Finding All Cliques of an Undirected Graph. *Commun. ACM* 16, 9 (1973), 575–577. <https://doi.org/10.1145/362342.362367>
- [4] Lijun Chang and Lu Qin. 2018. Cohesive Subgraph Computation over Large Sparse Graphs. *Springer Series in the Data Sciences* (2018). <https://doi.org/10.1007/978-3-030-03599-0>
- [5] James Cheng, Yiping Ke, Ada Wai-Chee Fu, Jeffrey Xu Yu, and Linhong Zhu. 2011. Finding Maximal Cliques in Massive Networks by H-Graph. *ACM Trans. Database Syst.* 36, 4 (2011), 21:1–21:34. <https://doi.org/10.1145/2043652.2043654>
- [6] Apurba Das, Michael D. Shah, Shreyas Pai, and Sriram V. Pemmaraju. 2019. A Shared-Memory Parallel Algorithm for Maximal Clique Enumeration from Large Sparse Graphs. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 461–472. <https://doi.org/10.1109/IPDPS.2019.00056>
- [7] David Eppstein, Maarten Löffler, and Darren Strash. 2010. Listing All Maximal Cliques in Sparse Graphs in Near-Optimal Time. *Algorithms and Computation* (2010), 403–414. https://doi.org/10.1007/978-3-642-17517-6_36
- [8] David Eppstein and Darren Strash. 2013. Listing All Maximal Cliques in Large Sparse Real-World Graphs. *J. Exp. Algorithmics* 18 (2013), 3.1:1–3.1:21. <https://doi.org/10.1145/2543629>
- [9] Yixiang Fang, Reynold Cheng, Siqiang Luo, and Jiafeng Hu. 2017. Effective Community Search over Large Spatial Graphs. In *Proc. VLDB Endow.*, Vol. 10. 709–720. <https://doi.org/10.14778/3055330.3055343>
- [10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Trans. Big Data* 7, 3 (2021), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>
- [11] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. *Complexity of Computer Computations* (1972), 85–103.
- [12] Byron Lessley, Talita Perciano, and Hank Childs. 2016. Data-Parallel Techniques for High Performance Maximal Clique Enumeration. In *2016 IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA)*. 636–645. <https://doi.org/10.1109/DSAA.2016.78>
- [13] Rong-Hua Li, Lu Qin, Jeffrey Xu Yu, and Rui Mao. 2015. Influential Community Search in Large Networks. In *Proc. VLDB Endow.*, Vol. 8. 509–520. <https://doi.org/10.14778/2735479.2735484>
- [14] Yuchen Li, Yuxiang Zeng, Zhifeng Bao, Shirui Pan, J. Shane Culpepper, and Renjie Sun. 2022. Walking with Perception: Efficient Random Walk Sampling via Common Neighbor Awareness. *IEEE Trans. Knowl. Data Eng.* 34, 1 (2022), 152–164. <https://doi.org/10.1109/TKDE.2020.2981316>
- [15] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836. <https://doi.org/10.1109/TPAMI.2019.2887751>
- [16] Ryan A. Rossi, Rong Zhou, and Nesreen K. Ahmed. 2015. Parallel Maximum Clique Algorithms with Applications to Network Analysis and Storage. *SIAM J. Sci. Comput.* 37, 5 (2015), C589–C616. <https://doi.org/10.1137/141002081>
- [17] Etsuji Tomita, Akira Tanaka, and Haruhisa Takahashi. 2006. The Worst-Case Time Complexity for Generating All Maximal Cliques and Computational Experiments. *Theor. Comput. Sci.* 363, 1 (2006), 28–42. <https://doi.org/10.1016/j.tcs.2006.06.015>
- [18] Etsuji Tomita, Akira Tanaka, and Haruhisa Takahashi. 2006. The Worst-Case Time Complexity for Generating All Maximal Cliques and Computational Experiments. *Theor. Comput. Sci.* 363, 1 (2006), 28–42. <https://doi.org/10.1016/j.tcs.2006.06.015>
- [19] Kai Wang, Xin Cao, Xuemin Lin, Wenjie Zhang, and Lu Qin. 2020. Efficient Computing of Radius-Bounded k -Cores. *IEEE Trans. Knowl. Data Eng.* 32, 2 (2020), 235–248. <https://doi.org/10.1109/TKDE.2018.2890642>
- [20] Yang Zhou, Hong Cheng, and Jeffrey Xu Yu. 2009. Graph Clustering Based on Structural/Attribute Similarities. In *Proc. VLDB Endow.*, Vol. 2. 718–729. <https://doi.org/10.14778/1687627.1687709>